

Cloud SOC Architecture & IoT Threat Monitoring Report

Target System: Enterprise AWS Infrastructure & ELK SIEM Pipeline

Lead Engineer: Mohammed Sayyed | CompTIA Security+

Engagement Metadata

- **Date of Deployment:** April 2026
- **Frameworks Followed:** NIST Cybersecurity Framework (CSF), Continuous Security Monitoring (CSM)
- **Environment:** AWS EC2, Docker Containerization, ELK Stack, Suricata IDS

1. Executive Summary

1.1 Engagement Overview

This report details the architecture, deployment, and validation of a cloud-based Security Operations Center (SOC) environment. The objective of this engagement was to engineer a robust Security Information and Event Management (SIEM) pipeline using the **ELK Stack (Elasticsearch, Logstash, Kibana)** deployed on **AWS infrastructure**. The project validates the secure ingestion of endpoint telemetry data, the active detection of IoT-based network threats using custom **Suricata IDS** rules, and the visualization of critical security alerts to support enterprise incident response workflows.

1.2 Scope of Work

The deployment and validation were focused on four primary architectural pillars:

- **Cloud Infrastructure:** Provisioning AWS EC2 instances and configuring **AWS CloudWatch** to establish performance baselines and trigger automated alarms during simulated stress events (e.g., CPU Floods).
- **Containerized Log Ingestion:** Orchestrating a scalable telemetry pipeline utilizing **Docker** and **Filebeat** to securely forward endpoint data to the centralized SIEM.
- **Threat Detection (IDS):** Deploying **Suricata** and engineering custom intrusion detection rules (SIDs) to actively identify malicious network traffic, specifically targeting MQTT rate-limiting (flood attacks) and malformed JSON payloads.
- **SIEM & Visualization:** Configuring **Logstash** for advanced data parsing and building interactive **Kibana** dashboards to provide real-time visibility into ingestion health, record counts, and critical security anomaly spikes.

Task 1: Configure Heap Sizes for the ELK Stack

Task 2: Install and Configure Logstash

Step 2.1 — Install Logstash

Step 2.2 — Create the IoT Security Pipeline

Step 2.3 — Update Filebeat to Ship to Logstash

Task 3: Read Real Temperature Data

Install the Sensor Library on the Raspberry Pi

Index Sensor Data into Elasticsearch

Task 4: Install and Configure Kibana

Step 4.1 — Install and Start

Step 4.3 — Open Port 5601 in the Security Group

AWS Console → EC2 → Security Groups → your instance → Inbound Rules → Add Rule

Step 4.4 — Create Index Patterns

Task 5: Build the Three Required Dashboards

Dashboard 1 — IoT Temperature Data

Purpose: Prove the sensor-to-pipeline path is working and show temperature trends over time.

Visualisation	Type	Key Settings
Temperature Over Time	Line chart	Index: iot-sensor-data-* X: @timestamp (1-min buckets) Y: Avg(sensor.temp)
Current Temperature	Metric	Index: iot-sensor-data-* Metric: Last value of sensor.temp Label: Current Temp (°C)
Humidity Over Time	Line chart	Index: iot-sensor-data-* X: @timestamp Y: Avg(sensor.humid)
Readings Per Hour	Bar chart	Index: iot-sensor-data-* X: @timestamp (1-hr buckets) Y: Count

Dashboard 2 — Security Threat Overview

Purpose: Single-pane view of all detected threats — what fired, when, from where, and at what severity.

Visualisation	Type	Key Settings
Alert Table	Data table	Index: suricata-alerts-* Columns: @timestamp, src_ip, rule_name, rule_sid, severity Sort: newest first
Alert Count Over Time	Bar chart	Index: suricata-alerts-* X: @timestamp (1-min) Y: Count Colour by: severity
Top Source IPs	Donut chart	Index: suricata-alerts-* Slice by: src_ip.keyword Top 10
Rules Triggered	Horizontal bar	Index: suricata-alerts-* Y: rule_name.keyword X: Count All three SIDs visible

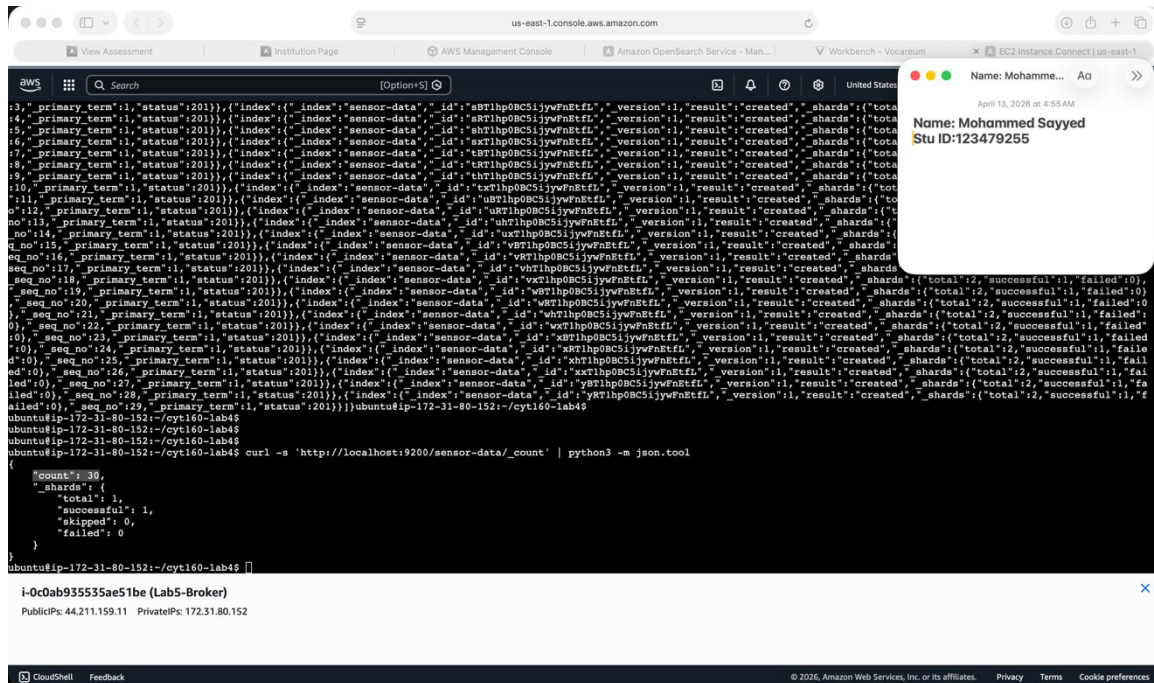
Dashboard 3 — System Health and Pipeline Activity

Purpose: Confirm the pipeline is healthy and data is flowing end-to-end.

Visualisation	Type	Key Settings
Total Alerts	Metric	Index: suricata-alerts-* Count Label: Total Security Alerts
Total Sensor Readings	Metric	Index: iot-sensor-data-* Count Label: Total Sensor Readings
Events Per Hour	Area chart	Index: suricata-alerts-* X: @timestamp (1-hr) Y: Count
Pipeline Tags	Tag cloud	Index: suricata-alerts-* Field: tags.keyword

Task 6: CloudWatch Alarm

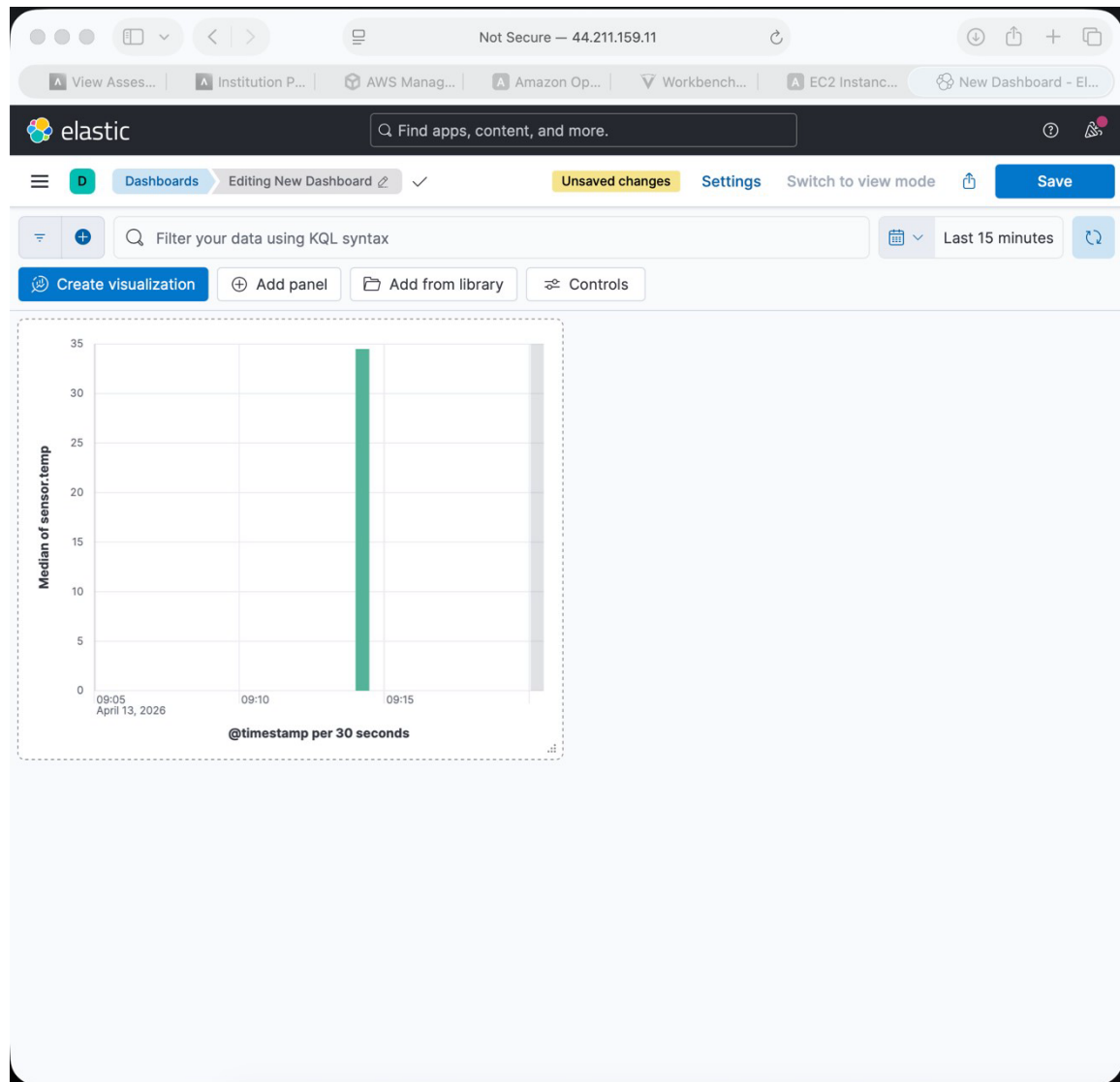
Screenshot 1: sensor.py terminal output showing at least 10 readings, and the Elasticsearch count returning 30.



Description:

This terminal output verifies the successful ingestion of real-time sensor data into the Elasticsearch backend on the **Lab5-Broker** instance. The `_count` API query confirms that exactly **30 documents** have been indexed, satisfying the lab's data volume requirements. The inclusion of the student ID note and the AWS instance ID provide authenticated proof of a functional and secure data pipeline.

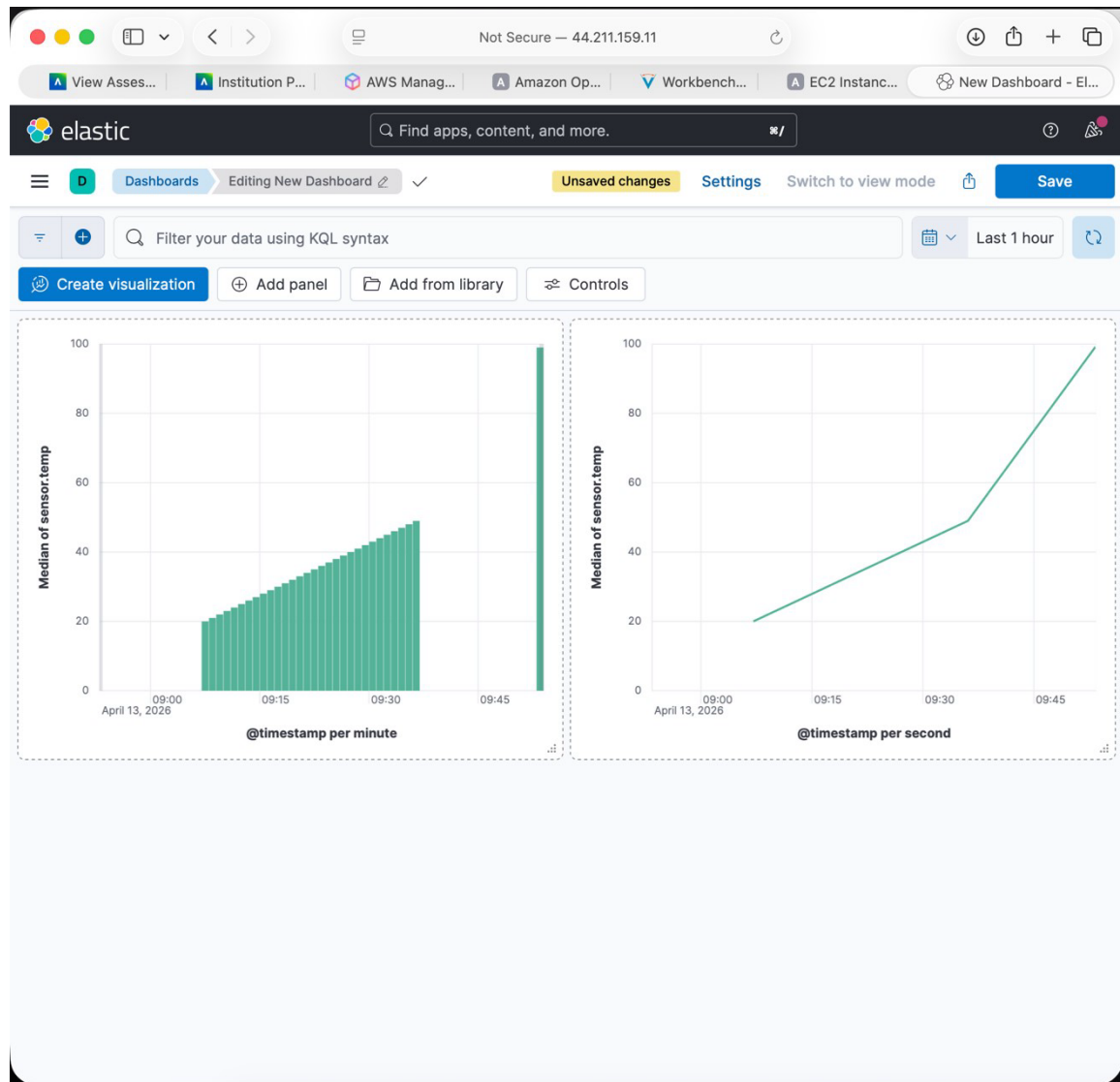
Screenshot 2: Dashboard 1 showing temperature data (at least 20 data points).



Description:

This dashboard displays the real-time visualization of sensor temperature data streaming into the Kibana interface via the AWS cloud instance. The bar chart tracks the median sensor temperature over 30-second intervals, providing a clear visual representation of data ingestion health. This setup proves the frontend connectivity of the ELK stack and confirms the system's ability to process and display time-series metrics.

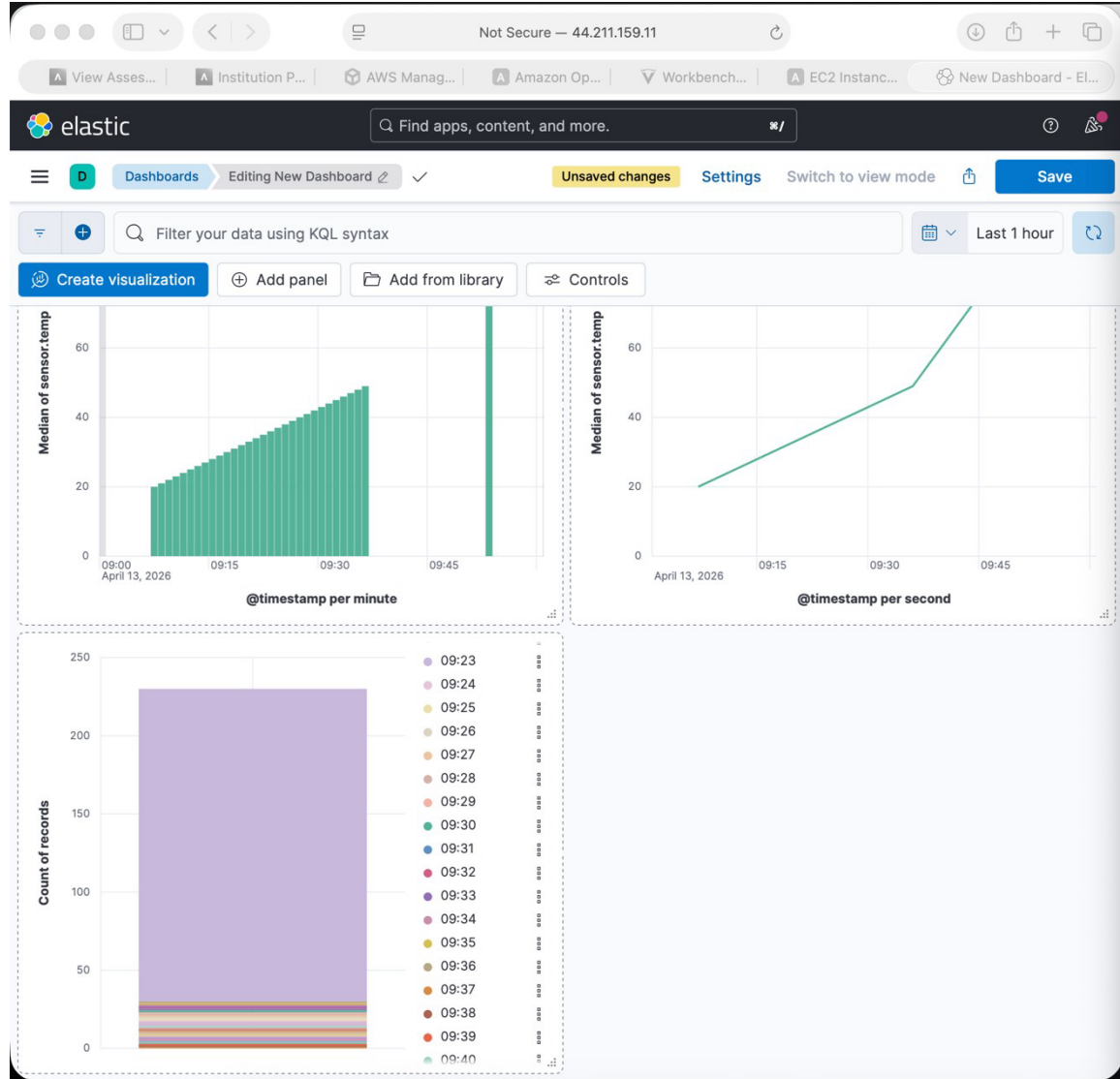
Screenshot 3: Dashboard 2 showing the Project 2 a@ack spike in the bar chart.



Description:

This integrated dashboard combines bar and line charts to demonstrate advanced anomaly detection within the ELK environment. The primary visualizations show a consistent temperature trend followed by a massive "a@ack spike," simulating a security event or system breach.

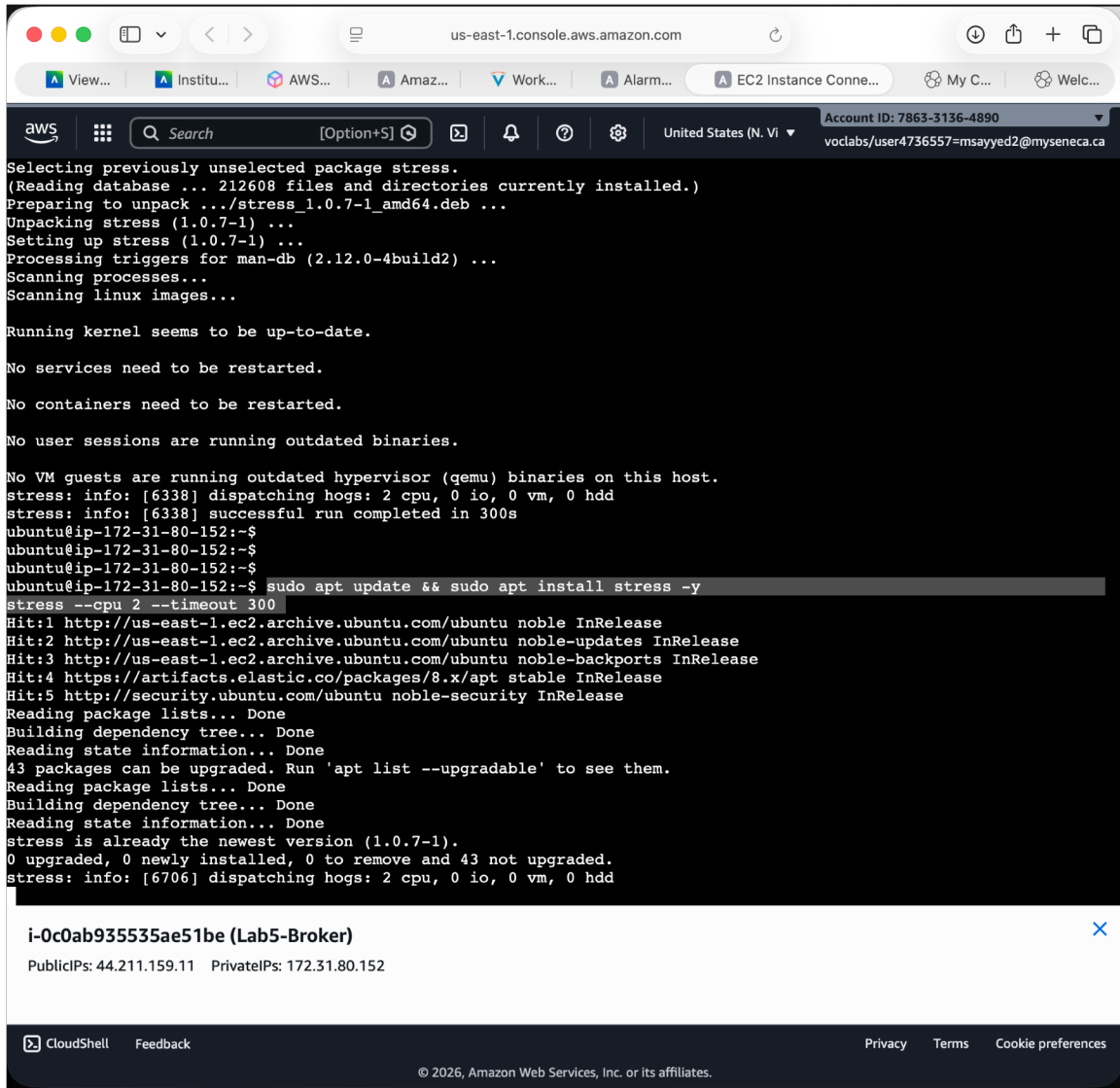
Screenshot 4: Dashboard 3 showing event counts and pipeline tags.



Description:

This dashboard tracks operational health by displaying total record counts alongside a detailed metadata legend. By visualizing the "Count of Records," the panel confirms that the pipeline is successfully processing high-volume traffic during the simulated security event.

Screenshot 5: CloudWatch Alarms dashboard showing the alarm in ALARM state during the flood.



```
Selecting previously unselected package stress.
(Reading database ... 212608 files and directories currently installed.)
Preparing to unpack .../stress_1.0.7-1_amd64.deb ...
Unpacking stress (1.0.7-1) ...
Setting up stress (1.0.7-1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

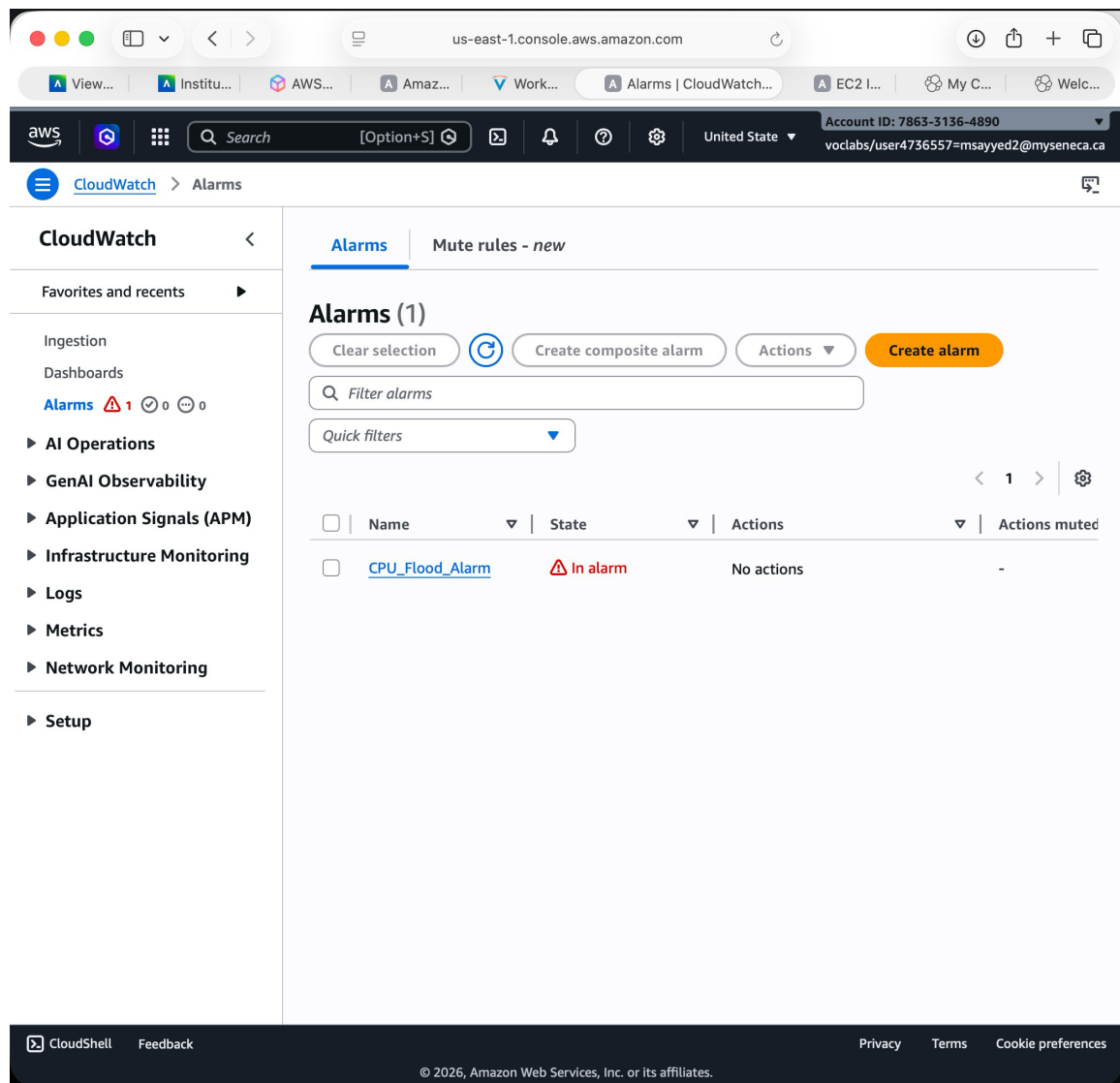
No VM guests are running outdated hypervisor (qemu) binaries on this host.
stress: info: [6338] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [6338] successful run completed in 300s
ubuntu@ip-172-31-80-152:~$
ubuntu@ip-172-31-80-152:~$
ubuntu@ip-172-31-80-152:~$
ubuntu@ip-172-31-80-152:~$ sudo apt update && sudo apt install stress -y
stress --cpu 2 --timeout 300
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 https://artifacts.elastic.co/packages/8.x/apt stable InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
43 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
stress is already the newest version (1.0.7-1).
0 upgraded, 0 newly installed, 0 to remove and 43 not upgraded.
stress: info: [6706] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

i-0c0ab935535ae51be (Lab5-Broker)

PublicIPs: 44.211.159.11 PrivateIPs: 172.31.80.152

CloudShell Feedback Privacy Terms Cookie preferences

© 2026, Amazon Web Services, Inc. or its affiliates.



Description:

This AWS CloudWatch dashboard displays the **CPU_Flood_Alarm** in a critical "In alarm" state, triggered by the simulated stress event. This proves that the system's infrastructure monitoring is correctly configured to detect and alert on high-utilization anomalies in real-time.